AWS VPC Peering

Private Connectivity Between Two VPCs

Student Project Report | AWS Architect Associate

Student Name	To be completed
Batch / Class	To be completed
AWS Region	Planned: ap-south-1 (Mumbai); replace if different
Completion Date	To be completed

Submission note: this report is a build-and-test report draft. It does not claim that AWS resources were deployed or tests were run. Replace every marked evidence field with your own AWS IDs, terminal output and screenshots before submitting as completed work.

Contents

1 Executive Summary 1
2 Objectives and Scope 1
2.1 Learning objectives 1
2.2 Scope and assumptions 1
3 Architecture and Address Plan 2
3.1 CIDR allocation 2
4 AWS Resources and Configuration 2
4.1 VPCs, subnets and Internet Gateway 2
4.2 EC2 instances 3
4.3 Security group ingress rules 3
5 Peering and Routing Plan 3
5.1 Required route tables 3
6 Administration and Test Procedure 4
6.1 SSH access from the student's laptop 4
6.2 Acceptance test matrix 5
7 Troubleshooting Notes 5
8 Security, Limitations and Cost Awareness 6
9 Cleanup Checklist 6
10 Student Completion Record 7
10.1 What I learned 7
10.2 Submission evidence checklist 7

1 Executive Summary

Is project ka goal AWS ke do alag Virtual Private Clouds (VPCs) ko VPC Peering se privately connect karna hai. VPC A mein ek public bastion host aur ek private application server hai. VPC B mein doosra private application server hai. App-A aur App-B ko apne private IPv4 addresses se communicate karna chahiye; dono application servers ko public IPv4 address nahi diya jana chahiye.

Project mein custom subnets, teen explicit route-table associations, security-group restrictions, peering connection, bidirectional routes, bastion ke through SSH, ping tests, ek controlled ICMP failure, aur cleanup shamil hain. Peering link akela connectivity guarantee nahi karta: dono taraf ke routes aur destination security rules bhi sahi hone chahiye.

Project status in this document: design and configuration plan prepared from the supplied workbook. Actual AWS resource IDs, test output and screenshots are intentionally left for the student's own run; expected results are not presented as observed results.

2 Objectives and Scope

2.1 Learning objectives

- Non-overlapping CIDR ranges plan karna aur VPC/subnet relationship samajhna.
- Public aur private subnets create karke route tables explicitly associate karna.
- Same-account, same-Region peering request create, accept aur Active status verify karna.
- Dono VPCs mein forward aur return routes configure karna.
- SSH ko bastion ke through route karna aur private hosts par public IP exposure rokna.
- Positive ping/SSH tests ke saath controlled negative test aur recovery demonstrate karna.
- Lab ke baad chargeable resources remove karke cleanup verify karna.

2.2 Scope and assumptions

Yeh design ek AWS account aur ek Region ke liye hai. Workbook Amazon Linux 2023, EC2 key pair, local OpenSSH client, default allow-all network ACLs aur lab-approved small instance type (misal: t3.micro) assume karta hai. VPC B mein Internet Gateway ya NAT Gateway nahi banaya jayega. Instance type, pricing aur account credits launch se pehle AWS console mein verify karne hain.

3 Architecture and Address Plan

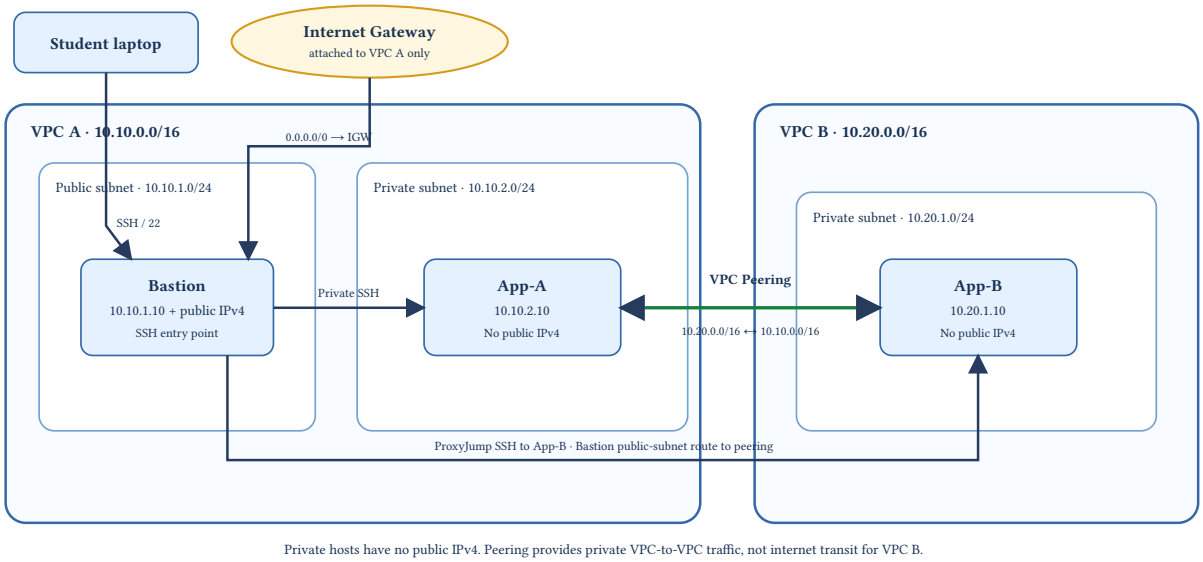


图 1 Workbook ke address plan par aadharit private VPC peering architecture.

3.1 CIDR allocation

Resource	Network	CIDR / IP	Role
VPC A	—	`10.10.0.0/16`	Bastion + App-A
Public subnet A	VPC A	`10.10.1.0/24`	Bastion
Private subnet A	VPC A	`10.10.2.0/24`	App-A
VPC B	—	`10.20.0.0/16`	App-B workload
Private subnet B	VPC B	`10.20.1.0/24`	App-B
Bastion	Public subnet A	`10.10.1.10` + public IPv4	Admin entry
App-A	Private subnet A	`10.10.2.10`	No public IPv4
App-B	Private subnet B	`10.20.1.10`	No public IPv4

Dono VPC CIDR ranges overlap nahi karte. Isliye unke beech peering request ban sakti hai. Workbook ke supplied sketch mein `28.x.x.x` ranges dikhte hain; woh is project ke written address plan se match nahi karte. Is report mein workbook ka consistent plan—`10.10.0.0/16` aur `10.20.0.0/16`—use kiya gaya hai.

4 AWS Resources and Configuration

4.1 VPCs, subnets and Internet Gateway

VPC A (`ewb-vpc-a`) mein `ewb-a-public` aur `ewb-a-private` subnets banenge. `ewb-igw-a` ko VPC A se attach kiya jayega. Public route table `ewb-rt-a-public` ko public subnet se explicitly associate karke `0.0.0.0/0 → ewb-igw-a` route add hoga. Private route table `ewb-rt-a-private` sirf private subnet se associate hoga; peering route add hone tak ismein VPC ka local route hi rahega. Private subnet mein auto-assign public IPv4 disabled rahega.

VPC B (ewb-vpc-b) mein sirf private subnet ewb-b-private aur route table ewb-rt-b-private banenge. Is subnet ko explicitly associate kiya jayega. VPC B mein Internet Gateway/NAT Gateway nahi hoga aur public IPv4 auto-assignment disabled hoga.

4.2 EC2 instances

- ewb-bastion: VPC A public subnet, private IP 10.10.1.10, public IPv4 enabled.
- ewb-app-a: VPC A private subnet, private IP 10.10.2.10, public IPv4 disabled.
- ewb-app-b: VPC B private subnet, private IP 10.20.1.10, public IPv4 disabled.

Teeno instances ke liye lab key pair ewb-lab use hoga. Private .pem key sirf student ke local computer par rahegi; use instance par upload ya report mein include nahi karna. Amazon Linux 2023 ka default SSH username ec2-user hai.

4.3 Security group ingress rules

Security group	Protocol	Source	Purpose
ewb-sg-bastion	TCP 22 / SSH	Student current public IPv4 /32	Laptop to bastion
ewb-sg-app-a	TCP 22 / SSH	10.10.1.10/32	Bastion to App-A
ewb-sg-app-a	ICMPv4 Echo Request, type 8/ code 0	10.20.1.10/32	App-B to App-A ping
ewb-sg-app-b	TCP 22 / SSH	10.10.1.10/32	Bastion to App-B
ewb-sg-app-b	ICMPv4 Echo Request, type 8/ code 0	10.10.2.10/32	App-A to App-B ping

SSH ka port 22 Port field mein set hota hai; source IP alag Source field mein diya jata hai. App-B tak ProxyJump ke zariye pahunchte waqt source Bastion ka private IP hota hai. Workbook ke beginner lab mein default outbound allow-all security-group rules aur default network ACLs rakhne hain. SSH ya ICMP ko 0.0.0.0/0 se open nahi karna.

5 Peering and Routing Plan

Peering connection ka naam ewb-a-b-peer hoga. Requester VPC A aur accepter VPC B select honge. Same account/Region hone par bhi request accept karna zaroori hai. Routes add karne se pehle status Active verify karke actual pcx-... ID note karni hai.

5.1 Required route tables

Route table	Destination	Target	Reason
ewb-rt-a-private	10.10.0.0/16	local	VPC A internal traffic
ewb-rt-a-private	10.20.0.0/16	Peering pcx-...	App-A → VPC B
ewb-rt-b-private	10.20.0.0/16	local	VPC B internal traffic
ewb-rt-b-private	10.10.0.0/16	Peering pcx-...	Return path to VPC A

ewb-rt-a-public	0.0.0.0/0	Internet Gateway	Bastion internet entry/ egress path
ewb-rt-a-public	10.20.0.0/16	Peering pcx-...	Bastion ⟶ App-B

Har subnet ke liye uska sahi route table association verify karna hai. Return route ke bina App-B ka response App-A tak wapas nahi aa sakta. Bastion public subnet mein peering route isliye chahiye kyunki ProxyJump mein Bastion destination App-B ko separately connect karta hai. Peering VPC B ko Internet Gateway transit ya VPC A ke zariye internet access nahi deta.

6 Administration and Test Procedure

6.1 SSH access from the student's laptop

`.pem` key ko local machine ke `~/.ssh/ewb-lab.pem` mein rakho aur permissions restrict karo:

```
chmod 400 ~/.ssh/ewb-lab.pem
```

`~/.ssh/config` mein actual bastion public IPv4 confirm karke yeh entries set karo:

```
Host ewb-bastion
  HostName BASTION_PUBLIC_IP
  User ec2-user
  IdentityFile ~/.ssh/ewb-lab.pem
  IdentitiesOnly yes

Host ewb-app-a
  HostName 10.10.2.10
  User ec2-user
  IdentityFile ~/.ssh/ewb-lab.pem
  IdentitiesOnly yes
  ProxyJump ewb-bastion

Host ewb-app-b
  HostName 10.20.1.10
  User ec2-user
  IdentityFile ~/.ssh/ewb-lab.pem
  IdentitiesOnly yes
  ProxyJump ewb-bastion
```

Pehle `ssh ewb-bastion`, phir laptop se `ssh ewb-app-a` aur routes/peering ke baad `ssh ewb-app-b` test karo. SSH commands laptop par chalti hain; key bastion par copy nahi hoti.

6.2 Acceptance test matrix

Important: neeche ka "Expected" column workbook ke acceptance criteria batata hai, actual observation nahi. "Actual result" aur "Evidence" tabhi fill hon jab student ne apne AWS lab mein command run karke screenshot/output record kiya ho.

Test	Command / location	Expected criterion	Actual result: fill after run	Evidence file / screenshot
Baseline	App-A: `ping -c 4 10.20.1.10`, before peering/routes	No replies	PENDING	Paste own screenshot
Forward ping	App-A: `ping -c 4 10.20.1.10`, after routes	4 replies, 0% loss	PENDING	Paste own screenshot
Reverse ping	App-B: `ping -c 4 10.10.2.10`	4 replies, 0% loss	PENDING	Paste own screenshot
Private SSH	Laptop: `ssh ewb-app-b`	Shell on App-B via bastion	PENDING	Paste own screenshot
Isolation	EC2 instance details	App-A/App-B have no public IPv4	PENDING	Paste own screenshot
Controlled failure	Temporarily remove only App-B ICMP Echo Request from App-A `/32`; repeat forward ping	Ping replies stop; SSH rule stays	PENDING	Paste failure screenshot
Recovery	Restore exact ICMP rule; repeat forward ping	Replies resume	PENDING	Paste recovery screenshot

On each private host, `hostname`, `hostname -I` aur `ping -c 4 PEER_PRIVATE_IP` chala kar host identity aur private destination verify karo. Pehli ping mein host readiness ke kaaran packet loss aaye toh thoda rukkar test repeat karo. Ping ke liye source ke bajay destination host par ICMP Echo Request rule allow hona chahiye.

7 Troubleshooting Notes

- **Bastion SSH timeout:** bastion ka current public IP, student public IP `/32` ingress rule, public subnet association aur Internet Gateway default route check karo.
- **App-A SSH failure:** private IP, bastion source `10.10.1.10/32`, key pair, `ec2-user`, instance state aur status checks verify karo.
- **App-B SSH failure:** VPC A public route table ka `10.20.0.0/16 → pcx` route, VPC B ka return route aur App-B SSH ingress source check karo.
- **Ping fails although peering Active:** dono private route tables, correct `pcx` target, subnet associations, ICMP rule aur actual instance IPs check karo.
- **Peering create nahi hoti:** dono VPC ke saare associated CIDRs (secondary CIDRs samet) non-overlapping hone chahiye.

- **`Permission denied (publickey):`** network path SSH authentication tak pahunch gaya ho sakta hai; correct key, local `IdentityFile` aur username verify karo.

8 Security, Limitations and Cost Awareness

VPC Peering one-to-one aur non-transitive hai. A-B aur A-C peering se B-C automatic connect nahi hota. Is design mein peering private IPv4 traffic ke liye hai; VPC B ko VPC A ke Internet Gateway ke zariye internet access nahi milta. Security groups stateful hote hain; permitted connection ke response traffic ko automatically allow karte hain. Custom network ACL use karne par return traffic ke liye extra rules chahiye, isliye yeh lab default ACLs par rehta hai.

EC2 compute, EBS volumes aur public IPv4 usage par AWS charges lag sakte hain. Launch se pehle Region ki current pricing aur account credits check karo. NAT Gateway is lab ke liye required nahi hai. Kaam poora hone par instances terminate karna aur leftover chargeable resources check karna zaroori hai.

9 Cleanup Checklist

Evidence save karne ke baad sirf project ke resources delete karo:

1. `ewb-bastion`, `ewb-app-a`, `ewb-app-b` terminate karo; lab EBS volumes/snapshots aur banayi hui Elastic IP release/delete verify karo.
2. Peering routes hatao, `ewb-a-b-peer` delete karo; ENI disappear hone ke baad lab security groups delete karo.
3. Zaroorat par subnet associations remove karo; lab subnets aur custom route tables delete karo.
4. `ewb-igw-a` detach karke delete karo, phir VPC A aur VPC B delete karo.
5. Project tag se Region mein EC2, EBS, public IPv4 allocations aur VPC resources dobara inspect karo. Billing data late aa sakta hai.
6. Unused lab key pair delete karo; local `.pem` ko bhi remove karo agar future mein zaroorat na ho. Pehle evidence save karna.

10 Student Completion Record

Actual run ke baad yeh fields bharo. Resource IDs aur observed outcomes AWS console/terminal se hi copy karo.

Record	Student's actual value
AWS Region	To be completed
VPC A ID / VPC B ID	To be completed
Peering ID and status	To be completed
Bastion instance ID / public IPv4	To be completed
App-A / App-B instance IDs	To be completed
Actual completion date	To be completed

10.1 What I learned

Student: apne actual experience ke aadhar par 3–5 sentences likhe. Suggested prompts: return route ka role kya tha? Active peering ke bawajood connectivity kin controls par depend karti hai? Bastion private workloads ko public IP diye bina administration kaise deta hai? Controlled failure se aapne kaunsa security rule verify kiya?

10.2 Submission evidence checklist

- [ ] Actual architecture/resource IDs and IP inventory attached.
- [ ] All three route tables and subnet associations shown.
- [ ] Peering `Active` screenshot and actual connection ID recorded.
- [ ] Before/after ping outputs and private SSH-through-bastion output attached.
- [ ] ICMP rule removal failure and restoration success shown.
- [ ] App-A and App-B public IPv4 fields shown empty.
- [ ] Cleanup evidence saved; actual learning reflection completed.
- [ ] No private key, password or access token included.